GUIA DE ENTREVISTA BACKEND .NET

C# • ASP.NET Core • Clean Architecture
SOLID • Design Patterns • REST • EF Core
SignalR • Docker • Exercícios práticos

Edição reorganizada — Outubro de 2026

Material de preparação e partilha entre colegas

Índice

Como utilizar este guia **1**

1 C# e LINQ — fundamentos **2**

1.1 Conceitos principais . . . 2

1.2 LINQ e execução diferida . . . 2

1.2.1 LINQ . . . 2

1.3 Complexidade: o que convém saber . . . 3

1.4 Perguntas de entrevista — Português e English . . . 3

2 POO e princípios SOLID **4**

2.1 Os quatro pilares da POO . . . 4

2.1.1 Encapsulamento (Encapsulation) . . . 4

2.1.2 Abstração (Abstraction) . . . 4

2.1.3 Herança (Inheritance) . . . 4

2.1.4 Polimorfismo (Polymorphism) . . . 4

2.2 Os cinco princípios SOLID . . . 5

2.2.1 S — Single Responsibility Principle . . . 5

2.2.2 O — Open/Closed Principle . . . 5

2.2.3 L — Liskov Substitution Principle . . . 5

2.2.4 I — Interface Segregation Principle . . . 5

2.2.5 D — Dependency Inversion Principle . . . 5

2.3 Aplicação prática . . . 6

2.4 Perguntas de entrevista — Português e English . . . 6

3 Design Patterns e padrões arquiteturais **8**

3.1 Três categorias GoF . . . 8

3.2 Creational — criação de objetos . . . 8

3.3 Structural — composição de objetos . . . 8

3.4 Behavioral — comportamento e comunicação . . . 8

3.5 Exemplos em C# — Observer e Strategy . . . 9

3.6 Observer — exemplo em C# . . . 9

3.7 Strategy — exemplo em C# . . . 9

3.8 Aplicação ao sistema de agentes . . . 10

3.9 Aplicação ao exercício de arquitetura . . . 10

3.10 Outros padrões relevantes . . . 10
3.11 Principais estilos e padrões de arquitetura . . . 10
3.11.1 Layered / N-tier Architecture . . . 10
3.11.2 Clean Architecture . . . 10
3.11.3 Hexagonal / Ports and Adapters . . . 10
3.11.4 Modular Monolith . . . 11
3.11.5 Microservices . . . 11
3.11.6 Event-Driven Architecture . . . 11
3.11.7 CQRS . . . 11
3.11.8 Repository Pattern . . . 11
3.11.9 Unit of Work . . . 11
3.11.10 Outbox Pattern . . . 11
3.12 Perguntas de entrevista — Português e English . . . 11

4 ASP.NET Core — ciclo HTTP e middleware 13
4.1 Percurso do pedido HTTP . . . 13
4.2 1. Percurso completo: do pedido à resposta . . . 13
4.3 Pipeline em Program.cs . . . 13
4.4 O que faz next() e porque a resposta regressa em ordem inversa? . . . 14
4.5 3. Como funciona o next() e a ordem inversa da resposta . . . 14
4.6 HttpContext, model binding e DI . . . 14
4.7 4. HttpContext, controller e DI . . . 15
4.8 5. Middleware versus filters versus services . . . 15
4.9 Dependency Injection lifetimes . . . 15
4.10 Perguntas de entrevista — Português e English . . . 15

5 RESTful APIs, HTTP e segurança 17
5.1 REST e semântica HTTP . . . 17
5.2 1. O que significa RESTful? . . . 17
5.3 2. Métodos HTTP — tabela de referência . . . 17
5.4 Exemplos da Agent API . . . 17
5.5 3. Exemplos com a Agent API . . . 17
5.5.1 GET /api/agents/team . . . 18
5.5.2 POST /api/agents . . . 18
5.5.3 PUT /api/agents/{id} . . . 18
5.5.4 PATCH /api/agents/me/status . . . 18
5.5.5 DELETE /api/agents/{id} . . . 18

5.5.6 HEAD /api/agents/{id} . . . 18
5.5.7 OPTIONS /api/agents . . . 18
5.6 4. PUT vs PATCH: a diferença essencial . . . 18
5.7 Códigos de resposta HTTP . . . 19
5.8 Exemplo de endpoints ASP.NET Core . . . 19
5.9 6. Exemplos ASP.NET Core . . . 19
5.10 Autenticação, autorização e JWT . . . 20
5.11 Perguntas de entrevista — Português e English . . . 20

6 EF Core, SQL, concorrência e desempenho 21
6.1 Fundamentos EF Core . . . 21
6.2 Consulta filtrada por equipa . . . 21
6.3 Consistência entre DB e eventos . . . 21
6.4 Perguntas de entrevista — Português e English . . . 22

7 Clean Architecture — guia aprofundado 23
7.1 Responsabilidades por camada . . . 23
7.2 1. Camadas e responsabilidades . . . 23
7.2.1 Domain . . . 23
7.2.2 Application . . . 23
7.2.3 Infrastructure . . . 23
7.2.4 WebAPI / Presentation . . . 23
7.3 Regra das dependências . . . 23
7.4 2. Regra das dependências . . . 23
7.5 Exemplo: interfaces e serviço C# . . . 24
7.6 Composition Root em Program.cs . . . 24
7.7 4. Dependency Injection e composition root . . . 24
7.8 5. Quando faz sentido e erros comuns . . . 25
7.9 Testabilidade e trade-offs . . . 25
7.10 Perguntas de entrevista — Português e English . . . 25

8 Caso prático — Agent Presence System 27
8.1 Enunciado e requisitos . . . 27
8.2 Componentes e decisões arquiteturais . . . 27
8.3 Fluxos de GET, PATCH e notificações . . . 27
8.3.1 Fluxo de leitura e atualização . . . 27
8.4 Modelo e contratos do exercício original . . . 28
8.4.1 Entidade e contratos C# . . . 28

8.5 Serviço de alteração de estado 28
8.5.1 Serviço: validação, persistência e notificação 28
8.6 SignalR Hub e ConnectionTracker 28
8.6.1 Hub: ligações e grupos 29
8.7 Notificador e DI 29
8.7.1 Notificador e registos DI 29
8.8 Casos de falha, escala e segurança 29
8.8.1 Pontos avançados para discutir 30
8.9 Divisão de tarefas por equipa 30
8.10 Pontos de atenção durante o live coding 30
8.11 Perguntas de entrevista — Português e English 30

9 Docker, containers e Kubernetes 32
9.1 Docker — image, container e Dockerfile 32
9.1.1 Docker — revisão rápida 32
9.2 Volumes, rede e orquestração 32
9.3 Perguntas de entrevista — Português e English 33

10 Live Coding — exercícios resolvidos 34
10.0.1 FizzBuzz 34
10.0.2 Reverse string 34
10.0.3 Palindrome 34
10.0.4 Find duplicates 34
10.0.5 Second-largest distinct value 35
10.0.6 Character frequency 35
10.0.7 Two Sum — return indices 35
10.0.8 LINQ — filtering and grouping 35
10.0.9 REST CRUD — exemplo de endpoints 36
10.1 Melhorias e casos limite a explicar 36
10.2 Perguntas de entrevista — Português e English 36

11 Estratégia de entrevista e revisão final 37
11.0.1 Perguntas iniciais em inglês 37
11.1 Roteiro de resposta para System Design 37
11.2 Checklist de revisão 37
11.37. Checklist final de preparação 38
11.48. Checklist de entrevista 38
11.5 Perguntas de entrevista — Português e English 38

11.6 Nota final sobre a origem do material 39

Como utilizar este guia

Leitura sugerida: começar por C# e POO, avançar para ASP.NET Core e persistência, depois aprofundar arquitetura e o cenário de agentes. No final de cada capítulo há perguntas e respostas nos dois idiomas. As perguntas são exemplos de treino; não constituem uma lista verificada de perguntas de uma empresa específica.

Bloco	Capítulos	Objetivo
Fundamentos	1–3	C#, coleções, POO, SOLID e padrões
Backend .NET	4–6	ASP.NET Core, REST, EF Core e segurança
Arquitetura e sistema	7–8	Clean Architecture e Agent Presence System
Operação e prática	9–11	Docker, live coding e estratégia de entrevista

C# e LINQ — fundamentos

Tipos, coleções, expressões LINQ, assíncrono e complexidade.

Conceitos principais

- Value types (int, bool, struct) guardam valores; reference types (class, string, arrays) têm semântica de referência. string é imutável.
- Overloading: mesmo nome, assinaturas diferentes. Overriding: redefinição de método virtual numa classe derivada.
- Interface define contrato; classe abstrata pode conter implementação partilhada e estado. Ambas podem facilitar polimorfismo.
- async/await liberta a thread enquanto espera por I/O assíncrono; não torna operações CPU-bound automaticamente paralelas. Task representa operação, Task<T> representa também um resultado.
- using/await using ajudam a libertar recursos IDisposable/IAsyncDisposable. GC gere memória, não substitui libertação determinística de recursos.
- List<T> serve sequências ordenadas; Dictionary<TKey,TValue> mapeia chaves únicas para valores; HashSet<T> guarda elementos únicos com procura média O(1).
- Any(predicate) termina na primeira correspondência; Count(predicate) conta todas as correspondências numa sequência em memória.
- First lança exceção se não existir correspondência; FirstOrDefault devolve default; SingleOrDefault lança exceção quando há mais de uma correspondência.

LINQ e execução diferida

LINQ

```
var names = products.Where(p => p.Price > 100)
                    .OrderBy(p => p.Price)
                    .Select(p => p.Name)
                    .ToList();
```

LINQ costuma usar execução diferida até à enumeração. Em EF Core, evitar N+1, filtrar na base de dados, projetar DTOs e usar índices adequados.

```
var result = products.Where(p => p.Price > 100)
                     .OrderBy(p => p.Price)
                     .Select(p => p.Name)
                     .ToList();
```

IEnumerable<T> descreve uma sequência que pode ser enumerada; IQueryable<T> guarda uma expressão que um provider, como EF Core, pode traduzir para SQL. Nem todo o LINQ é executado em memória.

Complexidade: o que convém saber

Operação	Complexidade típica	Notas
List<T>.Contains	O(n)	Procura linear
HashSet<T>.Contains	O(1) média	Pode degradar em casos patológicos
Dictionary<TKey,TValue> lookup	O(1) média	Hashing
List<T>.Sort	O(n log n)	Depende da implementação
Any(predicate)	O(1) melhor; O(n) pior	Pára ao encontrar o primeiro

Perguntas de entrevista — Português e English

PT Quando preferes um HashSet a uma List?

Resposta: Quando preciso de valores únicos e testes de pertença eficientes.

EN When would you prefer a HashSet over a List?

Answer: When I need unique values and efficient membership checks.

PT Qual é a diferença entre Where e Select?

Resposta: Where filtra elementos; Select projeta ou transforma os resultados.

EN What is the difference between Where and Select?

Answer: Where filters elements; Select projects or transforms them.

PT FirstOrDefault e SingleOrDefault são iguais?

Resposta: Não. SingleOrDefault exige no máximo um resultado e lança exceção se houver vários.

EN Are FirstOrDefault and SingleOrDefault equivalent?

Answer: No. SingleOrDefault expects at most one match and throws if multiple matches exist.

PT Para que serve async/await?

Resposta: Para trabalhar com operações assíncronas sem bloquear a thread durante esperas de I/O.

EN What is async/await used for?

Answer: It handles asynchronous operations without blocking a thread during I/O waits.

POO e princípios SOLID

Conceitos de desenho de objetos e qualidade de código aplicados à Agent API.

Os quatro pilares da POO

Encapsulamento (Encapsulation)

Protege o estado interno de um objeto e controla como ele é alterado. Exemplo: um método ChangeStatus valida a transição, em vez de permitir alterações arbitrárias à propriedade Status.

Interview question: How do you prevent invalid changes to an object's state?

Abstração (Abstraction)

Expõe o comportamento essencial sem obrigar o consumidor a conhecer a implementação. Exemplo: IAgentRepository define GetByIdAsync sem revelar se utiliza EF Core ou armazenamento em memória.

Interview question: What is the difference between abstraction and encapsulation?

Herança (Inheritance)

Permite que uma classe derive de outra e reutilize ou especialize comportamento. Exemplo: AgentHub : Hub herda funcionalidades de SignalR. Evitar hierarquias desnecessárias; frequentemente é preferível composição.

Interview question: When would you choose composition over inheritance?

Polimorfismo (Polymorphism)

Permite utilizar implementações diferentes através de um contrato comum. Exemplo: IAgentStatusNotifier pode ter uma implementação SignalR e outra de teste.

Interview question: How does dependency injection benefit from polymorphism?

Exemplo de encapsulamento:

```
public class Agent
{
    public Guid Id { get; private set; }
    public AgentStatus Status { get; private set; }

    public void ChangeStatus(AgentStatus newStatus)
    {
```

```
        if (newStatus == AgentStatus.Offline)
            throw new InvalidOperationException("Offline is system-managed.");
        if (Status == newStatus) return;
        Status = newStatus;
    }
}
```

Os cinco princípios SOLID

S — Single Responsibility Principle

Uma classe deve ter uma razão principal para mudar. AgentService gere regras; AgentRepository persiste dados; SignalRAgentStatusNotifier envia notificações.

O — Open/Closed Principle

Componentes devem permitir extensão sem exigir alterações constantes ao código existente. Exemplo: adicionar EmailNotifier implementando IAgentStatusNotifier.

L — Liskov Substitution Principle

Uma implementação deve poder substituir o seu contrato sem quebrar as expectativas do consumidor. Um repositório em memória deve respeitar as mesmas regras observáveis que o repositório SQL.

I — Interface Segregation Principle

Preferir interfaces pequenas e específicas a interfaces enormes. Um serviço só de leitura não deve depender de métodos de escrita que nunca utiliza.

D — Dependency Inversion Principle

Código de alto nível depende de abstrações, não de detalhes concretos. AgentService depende de IAgentRepository e IAgentStatusNotifier.

Exemplo de Dependency Inversion:

```
public class AgentService
{
    private readonly IAgentRepository _repository;
    private readonly IAgentStatusNotifier _notifier;

    public AgentService(IAgentRepository repository,
                        IAgentStatusNotifier notifier)
    {
        _repository = repository;
        _notifier = notifier;
    }
}
```

Pergunta frequente: 'Is dependency injection the same as dependency inversion?' Não. Dependency Inversion é um princípio de desenho; Dependency Injection é uma técnica que pode ajudar a aplicá-lo.

Aplicação prática

```
public class Agent
{
    public AgentStatus Status { get; private set; }

    public void ChangeManualStatus(AgentStatus newStatus)
    {
        if (newStatus == AgentStatus.Offline)
            throw new InvalidOperationException("Offline é automático.");
        Status = newStatus;
    }
}
```

Encapsulamento protege a regra; abstrações como IAgentRepository permitem trocar a implementação da persistência. Dependency Injection é uma técnica; Dependency Inversion é um princípio.

Perguntas de entrevista — Português e English

PT Quais são os quatro pilares da POO?

Resposta: Encapsulamento, abstração, herança e polimorfismo.

EN What are the four pillars of OOP?

Answer: Encapsulation, abstraction, inheritance, and polymorphism.

PT Como aplicarias SRP no sistema de agentes?

Resposta: Separaria serviço de negócio, repositório de dados e notificador SignalR.

EN How would you apply SRP to the agent system?

Answer: I would separate the business service, data repository, and SignalR notifier.

PT Dependency Injection é igual a Dependency Inversion?

Resposta: Não. DI é uma técnica; DIP exige que o código central dependa de abstrações.

EN Is Dependency Injection the same as Dependency Inversion?

Answer: No. DI is a technique; DIP is the principle of depending on abstractions.

PT O que defende o Open/Closed Principle?

Resposta: Preferir extensão do comportamento sem modificar constantemente código existente.

EN What does the Open/Closed Principle state?

Answer: Software should be open for extension but closed for modification.

Design Patterns e padrões arquiteturais

Distinguir padrões GoF, padrões de acesso a dados e estilos arquiteturais.

Três categorias GoF

Os padrões de desenho (Design Patterns) são soluções reutilizáveis para problemas recorrentes de desenho orientado a objetos. Os 23 padrões clássicos GoF dividem-se em creational, structural e behavioral. Não são o mesmo que SOLID nem que padrões arquiteturais.

Creational — criação de objetos

- Singleton: Uma instância por âmbito definido; em ASP.NET Core AddSingleton partilha a instância no mesmo contentor/processo, não entre réplicas.
- Factory Method: Delegar a criação de objetos a implementações especializadas.
- Abstract Factory: Criar famílias de objetos relacionados sem acoplar o cliente às classes concretas.
- Builder: Construir objetos complexos por etapas.

Structural — composição de objetos

- Adapter: Traduzir uma interface externa para a interface esperada pela aplicação.
- Decorator: Acrescentar logging, métricas ou caching sem alterar a classe decorada.
- Facade: Fornecer uma interface simples sobre um conjunto complexo de componentes.
- Proxy: Controlar o acesso a outro objeto, por exemplo para caching ou acesso remoto.

Behavioral — comportamento e comunicação

- Observer: Notificar subscritores quando um evento ocorre; em C# pode ser implementado com events/delegates.

- Strategy: Trocar algoritmos através de uma interface, como estratégias de distribuição de tickets.
- Command: Representar uma operação como objeto, facilitando filas, registo ou execução diferida.
- State: Delegar comportamento a objetos de estado; útil quando regras de transição se tornam complexas.
- Mediator: Centralizar a coordenação entre componentes, reduzindo dependências diretas.
- Chain of Responsibility: Encadear processadores; o pipeline de middleware ASP.NET Core é um exemplo relacionado.

Exemplos em C# — Observer e Strategy

Observer — exemplo em C#

No sistema de agentes, a mudança de estado pode gerar um evento consumido por vários observadores. O exemplo abaixo é local ao processo; SignalR distribui eventos a clientes remotos e não é automaticamente uma implementação do mesmo contrato GoF.

```
public class AgentStatusPublisher
{
    public event Action<Guid, AgentStatus>? StatusChanged;

    public void Publish(Guid id, AgentStatus status)
        => StatusChanged?.Invoke(id, status);
}

var publisher = new AgentStatusPublisher();
publisher.StatusChanged += (id, status) =>
    Console.WriteLine($"Agent {id}: {status}");
publisher.Publish(Guid.NewGuid(), AgentStatus.Away);
```

Strategy — exemplo em C#

```
public interface IAssignmentStrategy
{
    Guid SelectAgent(IReadOnlyList<Guid> availableAgents);
}

public sealed class FirstAvailableStrategy : IAssignmentStrategy
{
    public Guid SelectAgent(IReadOnlyList<Guid> agents)
        => agents.Count > 0 ? agents[0]
            : throw new InvalidOperationException("No agents available");
}
```

Aplicação ao sistema de agentes

Aplicação ao exercício de arquitetura

- Observer: AgentStatusChanged é comunicado aos clientes autorizados; não substituir a persistência por um evento volátil.
- Strategy: diferentes políticas de atribuição de tickets aos agentes (round-robin, menos ocupados).
- Repository: contrato de acesso a dados em Application e implementação EF Core em Infrastructure; não faz parte dos 23 GoF.
- Decorator: acrescentar logging e métricas ao IAgentService sem alterar as regras de negócio.
- State: útil se as regras Online/Away/Busy/Offline crescerem; para quatro estados simples, enum + validação é suficiente.
- Singleton: ConnectionTracker partilhado apenas por uma instância da API; não resolve presença distribuída.

Outros padrões relevantes

Repository abstrai o acesso a dados; Unit of Work coordena a persistência num âmbito transacional; Outbox permite guardar alterações e eventos atomicamente para posterior publicação. Não são todos padrões GoF.

Principais estilos e padrões de arquitetura

Layered / N-tier Architecture

Separação em apresentação, negócio e dados. Simples para CRUD; atenção ao acoplamento entre camadas.

Clean Architecture

Dependências apontam para o núcleo. Domain não depende de EF Core, API ou SignalR. Application define contratos; Infrastructure implementa persistência; WebAPI expõe endpoints e Hub.

Hexagonal / Ports and Adapters

A lógica de negócio comunica através de portas (interfaces); adapters implementam SQL, REST ou SignalR. Muito próximo da ideia de dependências para dentro da Clean Architecture.

Modular Monolith

Uma aplicação implantada como unidade, organizada por módulos e limites claros. Boa escolha inicial para cerca de 500 agentes; menos complexidade operacional que microservices.

Microservices

Serviços implantados independentemente, com fronteiras de negócio, contratos de rede e dados próprios quando apropriado. Úteis quando escala, autonomia de equipas e deploy independente compensam a complexidade distribuída.

Event-Driven Architecture

Componentes reagem a eventos, por exemplo AgentStatusChanged. Útil para desacoplamento e processamento assíncrono; exige atenção a entrega, duplicados e consistência eventual.

CQRS

Separa modelos/operações de leitura e escrita quando necessidades diferem significativamente. Não é necessário por defeito num CRUD pequeno.

Repository Pattern

Abstrai operações de persistência. IAgentRepository na Application; AgentRepository com EF Core na Infrastructure.

Unit of Work

Agrupa alterações numa unidade transacional. EF Core DbContext já fornece mecanismos de tracking e SaveChanges que frequentemente cumprem esse papel.

Outbox Pattern

Guarda a alteração e o evento na mesma transação; um processo publica depois o evento. Ajuda a evitar perder notificações quando a base de dados grava mas o envio falha.

Perguntas de entrevista — Português e English

PT Qual é a diferença entre Observer e Strategy?

Resposta: Observer notifica interessados; Strategy encapsula algoritmos intercambiáveis.

EN What is the difference between Observer and Strategy?

Answer: Observer notifies subscribers; Strategy selects interchangeable algorithms.

PT Quando usarias o State Pattern?

Resposta: Quando cada estado tem lógica e transições suficientemente complexas para justificar objetos específicos.

EN When would you use the State pattern?

Answer: When each state has complex, distinct behavior and transition rules.

PT O que é um Adapter?

Resposta: Um objeto que converte uma interface noutra esperada pelo cliente.

EN What is an Adapter?

Answer: It translates one interface into another expected by the client.

PT O Repository é um dos 23 GoF?

Resposta: Não; é um padrão de abstração do acesso a dados.

EN Is Repository one of the 23 GoF patterns?

Answer: No; it is a data-access abstraction pattern.

ASP.NET Core — ciclo HTTP e middleware

Do pedido à resposta: Kestrel, pipeline, autenticação, controller, serviço e retorno.

Percurso do pedido HTTP

1. Percurso completo: do pedido à resposta

1. O cliente envia PATCH /api/agents/me/status com JSON e credenciais.
2. Kestrel recebe a ligação HTTP, analisa o pedido e cria o HttpContext.
3. O pedido entra no pipeline configurado no Program.cs. Um middleware de exceções pode envolver os restantes.
4. HTTPS redirection (se necessário), routing e autenticação identificam o endpoint e o utilizador; autorização verifica permissões.
5. O endpoint selecionado invoca o controller. Model binding lê o JSON e cria o DTO; validação pode rejeitar dados inválidos.
6. O controller chama IAgentService. O serviço valida as regras e usa IAgentRepository.
7. AgentRepository usa EF Core/DbContext; EF Core comunica com a base de dados e SaveChangesAsync persiste a alteração.
8. Após a persistência, IAgentStatusNotifier pode publicar AgentStatusChanged para o grupo SignalR da equipa.
9. O controller devolve um resultado HTTP, por exemplo 204 No Content, 400, 401, 403 ou 404, conforme o caso.
10. A resposta atravessa os middlewares de volta, em ordem inversa à entrada; podem adicionar headers, logs ou métricas.
11. Kestrel envia o status, headers e body ao cliente. As notificações SignalR são mensagens separadas nas ligações existentes.

Pipeline em Program.cs

```
var builder = WebApplication.CreateBuilder(args);
builder.Services.AddControllers();
builder.Services.AddSignalR();
// Configurar AddAuthentication com o esquema JWT ou cookies válido.
builder.Services.AddAuthorization();
```

```
var app = builder.Build();
app.UseExceptionHandler(); // Requer configuração apropriada de tratamento de erros.
app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthentication();
app.UseAuthorization();
app.MapControllers();
app.MapHub<AgentHub>("/agentHub");
app.Run();
```

O que faz next() e porque a resposta regressa em ordem inversa?

3. Como funciona o next() e a ordem inversa da resposta

```
app.Use(async (context, next) =>
```

{

Console.WriteLine(”A: request entering”);

await next(context);

Console.WriteLine(”A: response leaving”);

});

```
app.Use(async (context, next) =>
```

{

Console.WriteLine(”B: request entering”);

await next(context);

Console.WriteLine(”B: response leaving”);

});

// Ordem observada:

// A: request entering

// B: request entering

// B: response leaving

// A: response leaving

Se um middleware não chamar next(), os seguintes não executam. Exemplo: Authorization pode terminar o pedido com 403; Authentication sem credenciais válidas pode levar a 401 quando o endpoint exige autenticação.

HttpContext, model binding e DI

4. HttpContext, controller e DI

HttpContext contém Request, Response, User, Items, RequestServices e CancellationToken da ligação. O ASP.NET Core resolve controllers e serviços pelo container DI. Model binding converte route/query/body em argumentos e DTOs; a validação com [ApiController] pode produzir 400 automaticamente para ModelState inválido.

5. Middleware versus filters versus services

- Middleware: atua no pipeline HTTP e pode abranger qualquer endpoint; ideal para exceções globais, logging, autenticação e CORS.
- Filters (MVC): executam em torno de ações/controllers e tratam preocupações específicas do MVC, como filtros de ação ou exceção.
- Application Service: executa regras de negócio e casos de uso; não deve conhecer HttpContext nem detalhes do transporte.
- SignalR Hub: gere comunicação em tempo real; as ligações ao Hub também atravessam middleware HTTP durante o estabelecimento e requests associados.

Dependency Injection lifetimes

Lifetime	Significado	Utilização típica
Transient	Nova instância por resolução	Serviços leves sem estado
Scoped	Uma instância por scope (normalmente pedido HTTP)	DbContext, services, repository
Singleton	Uma instância por contentor de DI	ConnectionTracker local ao processo

Evitar dependências Scoped capturadas por Singletons. O Hub é gerido pelo framework; não precisa de registo manual.

Perguntas de entrevista — Português e English

PT O que é middleware?

Resposta: Um componente do pipeline HTTP que pode inspecionar, modificar ou interromper o pedido.

EN What is middleware?

Answer: A component of the HTTP pipeline that can inspect, modify, pass on, or short-circuit a request.

PT Porque importa a ordem dos middlewares?

Resposta: Porque a autenticação deve estabelecer a identidade antes da autorização e o tratamento de exceções deve abranger os passos seguintes.

EN Why does middleware order matter?

Answer: Authentication should establish identity before authorization, and error handling should wrap downstream execution.

PT O que faz await next()?

Resposta: Executa o middleware seguinte; o código posterior corre no regresso da resposta.

EN What does await next() do?

Answer: It invokes the next middleware; code afterward runs while the response unwinds.

PT Qual é a diferença entre Scoped e Singleton?

Resposta: Scoped vive por scope; Singleton vive por contentor, devendo ser seguro para acesso concorrente.

EN What is the difference between Scoped and Singleton?

Answer: Scoped is per scope; Singleton is shared across the DI container and must handle concurrency safely.

RESTful APIs, HTTP e segurança

Semântica dos verbos, idempotência, códigos e controlo de acesso.

REST e semântica HTTP

1. O que significa RESTful?

REST (Representational State Transfer) é um estilo arquitetural. Numa API RESTful, os recursos são identificados por URIs e manipulados através da semântica HTTP. Os pedidos devem ser stateless: cada pedido contém a informação necessária para o servidor o processar. Usar verbos HTTP corretamente não basta, por si só, para cumprir todas as restrições REST.

2. Métodos HTTP — tabela de referência

Seguro (safe): o cliente não pede uma alteração de estado ao servidor. Idempotente: repetir o mesmo pedido tem o mesmo efeito pretendido no estado do servidor que executá-lo uma vez. Isto não significa receber sempre o mesmo código HTTP, nem ausência de logs, métricas ou outros efeitos incidentais.

Método	Seguro?	Idempotente?	Uso principal
GET	Sim	Sim	Ler recurso
HEAD	Sim	Sim	Ler metadados sem corpo
OPTIONS	Sim	Sim	Consultar opções
POST	Não	Não por definição	Criar ou invocar ação
PUT	Não	Sim	Substituir representação
PATCH	Não	Não garantido	Alterar parcialmente
DELETE	Não	Sim	Remover recurso

Exemplos da Agent API

3. Exemplos com a Agent API

GET /api/agents/team

Devolve os agentes da equipa do utilizador autenticado. Repetir não deve alterar o estado. Resposta típica: 200 OK.

POST /api/agents

Cria um novo agente. Repetir o pedido pode criar agentes duplicados; 201 Created com Location quando aplicável. Para evitar duplicação, considerar chave de idempotência ou identificador fornecido pelo cliente.

PUT /api/agents/{id}

Substitui a representação do agente nesse URI. Repetir com o mesmo payload deve deixar o recurso no mesmo estado pretendido. O contrato determina se a criação no URI é permitida.

PATCH /api/agents/me/status

Atualiza apenas o estado do agente autenticado. Exemplo: {"status":"Away"}. Uma operação "definir status = Away" pode ser desenhada como idempotente; uma operação "incrementar contador" não é necessariamente.

DELETE /api/agents/{id}

Remove o agente. O primeiro pedido pode devolver 204; o segundo pode devolver 404, mantendo a idempotência do efeito pretendido.

HEAD /api/agents/{id}

Consulta metadados/headers sem devolver corpo de resposta.

OPTIONS /api/agents

Consulta opções de comunicação; importante também nos preflight requests de CORS.

4. PUT vs PATCH: a diferença essencial

PUT destina-se a criar/substituir a representação do recurso no URI de destino, de acordo com o contrato. PATCH aplica alterações parciais. PATCH não é garantidamente idempotente pelo protocolo: depende da operação. Exemplos: "set status = Busy" é idempotente quanto ao estado; "increase score by 1" não é. Não confundas idempotência com o método devolver sempre o mesmo resultado.

Códigos de resposta HTTP

Código	Significado / quando usar
200 OK	Sucesso com conteúdo
201 Created	Recurso criado, idealmente com Location
204 No Content	Sucesso sem corpo
400 Bad Request	Entrada inválida
401 Unauthorized	Sem autenticação válida
403 Forbidden	Sem autorização
404 Not Found	Recurso não encontrado
409 Conflict	Conflito de estado ou unicidade
500 Internal Server Error	Erro inesperado do servidor

Exemplo de endpoints ASP.NET Core

6. Exemplos ASP.NET Core

GET com autorização de equipa:

```
[HttpGet("team")]
[Authorize]
public async Task<ActionResult<List<AgentDto>>> GetTeam(
    CancellationToken ct)
{
    var agents = await _service.GetMyTeamAgentsAsync(User, ct);
    return Ok(agents);
}
```

PATCH para alteração de estado:

```
public record UpdateStatusRequest(AgentStatus Status);

[HttpPatch("me/status")]
[Authorize]
public async Task<IActionResult> UpdateMyStatus(
    [FromBody] UpdateStatusRequest request,
    CancellationToken ct)
{
    await _service.UpdateMyStatusAsync(User, request.Status, ct);
    return NoContent();
}
```

Nota: os métodos de serviço são ilustrativos; o serviço deve obter a identidade validada, aplicar regras de autorização, persistir e só depois emitir a notificação SignalR. A conversão de enum para JSON deve ser configurada de acordo com o contrato.

Autenticação, autorização e JWT

Authentication identifica o utilizador; Authorization decide o que ele pode fazer. Um JWT tem header, payload e signature. Validar assinatura, issuer, audience e expiração; não confiar em TeamId ou AgentId arbitrários fornecidos pelo cliente.

CORS controla acessos por browser entre origens, não substitui autorização. Preferir DTOs para entrada/saída e códigos de erro consistentes (por exemplo ProblemDetails).

Perguntas de entrevista — Português e English

PT O que significa idempotência?

Resposta: Repetir o mesmo pedido tem o mesmo efeito pretendido no estado do servidor que executá-lo uma vez.

EN What does idempotent mean?

Answer: Repeating the same request has the same intended effect on server state as sending it once.

PT Que métodos são idempotentes?

Resposta: GET, HEAD, OPTIONS, PUT e DELETE; PATCH pode sê-lo conforme a operação.

EN Which HTTP methods are idempotent?

Answer: GET, HEAD, OPTIONS, PUT, and DELETE. PATCH may be idempotent depending on the operation.

PT Qual é a diferença entre PUT e PATCH?

Resposta: PUT substitui a representação do recurso; PATCH aplica alterações parciais.

EN What is the difference between PUT and PATCH?

Answer: PUT replaces the target resource representation; PATCH applies partial changes.

PT DELETE continua idempotente se responder 404 na segunda vez?

Resposta: Sim. As respostas podem diferir; importa o efeito pretendido no servidor.

EN Is DELETE idempotent if the second request returns 404?

Answer: Yes. Response codes can differ; idempotency concerns the intended effect on server state.

PT Qual é a diferença entre 401 e 403?

Resposta: 401 indica que falta autenticação válida; 403 indica acesso proibido.

EN What is the difference between 401 and 403?

Answer: 401 means valid authentication is missing; 403 means access is forbidden.

EF Core, SQL, concorrência e desempenho

Queries eficientes, tracking, transações, índices, migrations e consistência.

Fundamentos EF Core

- DbContext representa uma unidade de trabalho e é normalmente Scoped; não é thread-safe. Evitar operações simultâneas no mesmo DbContext.
- AsNoTracking() poupa tracking em queries apenas de leitura. Para atualizar uma entidade tracked, alterar a propriedade e chamar SaveChangesAsync() normalmente é suficiente.
- Projeções com Select evitam carregar colunas desnecessárias. Usar filtros e paginação no servidor.
- N+1 ocorre quando uma query principal desencadeia várias queries adicionais por entidade. Considerar projeção, Include quando necessário e inspeção do SQL.
- Índices aceleram filtros e joins, mas consomem armazenamento e acrescentam custo a escritas. Chaves primárias costumam estar indexadas.
- ACID: Atomicidade, Consistência, Isolamento e Durabilidade. Transações abrangem alterações que precisam de ser confirmadas juntas.
- Concorrência otimista usa tokens/versões para detetar atualizações concorrentes, normalmente com DbUpdateConcurrencyException. Pessimista usa bloqueios apropriados ao SGBD.
- Migrations versionam alterações de esquema; incluir migration, designer e snapshot no Git, mas não incluir bin/ nem obj/.

Consulta filtrada por equipa

```
var agents = await db.Agents
    .AsNoTracking()
    .Where(a => a.TeamId == authenticatedTeamId)
    .Select(a => new AgentDto(a.Id, a.Status))
    .ToListAsync(cancellationToken);
```

Consistência entre DB e eventos

Se SaveChangesAsync tem sucesso e o SignalR falha, o estado persistido continua válido, mas alguns clientes não são atualizados. Para exigências fortes de entrega,

implementar retries, reconciliação ou Transactional Outbox; tratar consumidores idempotentemente.

Perguntas de entrevista — Português e English

PT Para que serve AsNoTracking?

Resposta: Remove o custo do tracking em consultas apenas de leitura.

EN Why use AsNoTracking?

Answer: It avoids change-tracking overhead for read-only queries.

PT O que é um índice de base de dados?

Resposta: Uma estrutura que acelera consultas, com custo de armazenamento e escrita.

EN What is a database index?

Answer: A structure that speeds up queries but adds storage and write overhead.

PT O que é concorrência otimista?

Resposta: Detetar conflitos através de uma versão ou token, em vez de bloquear antecipadamente todas as alterações.

EN What is optimistic concurrency?

Answer: Detecting conflicts using a version or token instead of locking all updates in advance.

PT O que é o problema N+1?

Resposta: Uma query inicial provoca múltiplas queries adicionais, normalmente uma por elemento.

EN What is the N+1 query problem?

Answer: An initial query triggers many additional queries, often one per item.

Clean Architecture — guia aprofundado

Estrutura dos projetos, dependências, contrato versus implementação e testes.

Responsabilidades por camada

1. Camadas e responsabilidades

Domain

Entidades, value objects, enums e regras de negócio que não dependem de frameworks. Ex.: Agent, AgentStatus e invariantes de transição.

Application

Casos de uso, serviços, DTOs e contratos (interfaces). Ex.: AgentService, IAgentRepository e IAgentStatusNotifier.

Infrastructure

Implementações técnicas dos contratos, acesso a dados, DbContext, EF Core e integrações externas. Ex.: AgentRepository.

WebAPI / Presentation

Controllers, SignalR Hub, middleware, configuração de autenticação e composition root (Program.cs). Ex.: AgentsController, AgentHub e SignalRAgentStatusNotifier.

Regra das dependências

2. Regra das dependências

As dependências de código apontam para dentro: Domain não referencia Application, Infrastructure nem WebAPI; Application pode referenciar Domain, mas não EF Core nem SignalR; Infrastructure implementa interfaces da Application; WebAPI configura as implementações. O fluxo de execução em runtime pode atravessar as camadas no sentido inverso por meio das interfaces.

WebAPI → Application → Domain

Infrastructure → Application → Domain

Runtime: Controller → IAgentService → IAgentRepository → EF Core / SQL

└→ IAgentStatusNotifier → SignalR

Exemplo: interfaces e serviço C#

```
public interface IAgentRepository
{
    Task<Agent?> GetByIdAsync(Guid id, CancellationToken ct);
    Task SaveChangesAsync(CancellationToken ct);
}

public interface IAgentStatusNotifier
{
    Task NotifyAsync(Guid teamId, Guid agentId,
        AgentStatus status, CancellationToken ct);
}
```

```
public class AgentService
{
    private readonly IAgentRepository _repository;
    private readonly IAgentStatusNotifier _notifier;

    public AgentService(IAgentRepository repository, IAgentStatusNotifier notifier)
    {
        _repository = repository;
        _notifier = notifier;
    }

    public async Task<bool> UpdateAsync(Guid id, AgentStatus status, CancellationToken ct)
    {
        var agent = await _repository.GetByIdAsync(id, ct);
        if (agent is null) return false;
        // Validar transição e identidade autorizada antes de alterar.
        agent.Status = status;
        await _repository.SaveChangesAsync(ct);
        await _notifier.NotifyAsync(agent.TeamId, agent.Id, status, ct);
        return true;
    }
}
```

Composition Root em Program.cs

4. Dependency Injection e composition root

```
builder.Services.AddDbContext<AppDbContext>(options =>
```

options.UseSqlite(builder.Configuration.GetConnectionString(”Default”)));

```
builder.Services.AddScoped<IAgentRepository, AgentRepository>();
```

```
builder.Services.AddScoped<IAgentService, AgentService>();
```

```
builder.Services.AddScoped<IAgentStatusNotifier, SignalRAgentStatusNotifier>();
```

```
builder.Services.AddSingleton<ConnectionTracker>();
```

```
builder.Services.AddSignalR();
```

Scoped: normalmente um objeto por scope HTTP; Transient: novo por resolução; Singleton: uma instância por container de DI. Não injetar diretamente DbContext Scoped num Singleton.

5. Quando faz sentido e erros comuns

- Adequada quando o domínio tem regras e integrações que justificam isolamento; para uma API CRUD minúscula, evitar abstrações sem benefício.
- Não colocar consultas EF Core diretamente no controller nem IHubContext<AgentHub> no serviço Application.
- Repository deve concentrar persistência; Application coordena validações e casos de uso.
- SignalR Groups não substituem autorização nos endpoints HTTP.
- Clean Architecture não implica microservices: funciona muito bem num monólito modular.

Testabilidade e trade-offs

Para testar AgentService, substituir IAgentRepository e IAgentStatusNotifier por fakes/mocks. A aplicação não precisa de conhecer EF Core, SignalR ou HTTP. Não criar abstrações sem necessidade numa aplicação minúscula: Clean Architecture é um conjunto de limites de dependência, não uma obrigação de criar dezenas de projetos.

Perguntas de entrevista — Português e English

PT Onde deve ficar AgentHub?

Resposta: Na WebAPI/Presentation, porque trata comunicação com clientes.

EN Where should AgentHub live?

Answer: In the API/presentation layer because it handles client communication.

PT Onde deve ficar o repositório?

Resposta: Interface na Application; implementação EF Core na Infrastructure.

EN Where should the repository live?

Answer: Interface in Application; EF Core implementation in Infrastructure.

PT Qual é a regra central de dependências?

Resposta: Código exterior pode depender do interior; o domínio não depende de tecnologia externa.

EN What is the Dependency Rule?

Answer: Source-code dependencies point inward toward business rules.

PT Clean Architecture obriga a microservices?

Resposta: Não. Funciona tanto num monólito como num sistema distribuído.

EN Does Clean Architecture require microservices?

Answer: No. It works well in both monoliths and distributed systems.

Caso prático — Agent Presence System

Sistema de equipas, presença em tempo real, SignalR, múltiplas ligações e escala.

Enunciado e requisitos

Enunciado de treino: agentes de suporte consultam estados Online/Away/Busy/Offline dos colegas da própria equipa. Podem alterar o próprio estado para Online, Away ou Busy. Offline é automático após a última ligação. Há cerca de 500 agentes e são exigidas atualizações em tempo real.

Componentes e decisões arquiteturais

Escolha inicial: monólito modular com Clean Architecture e ASP.NET Core. A base de dados SQL guarda agentes e equipas; o serviço de presença controla estados e ligações; SignalR propaga alterações à equipa autorizada. Redis é opção quando é necessária coordenação entre instâncias.

Fluxos de GET, PATCH e notificações

Fluxo de leitura e atualização

- GET /api/agents: obter snapshot dos colegas da equipa autenticada (filtragem no servidor).
- PATCH /api/agents/me/status: validar identidade, estado permitido e transição; persistir via repository.
- Após persistência, IAgentStatusNotifier publica AgentStatusChanged para o grupo team-{id}.
- SignalR envia alterações aos clientes conectados; não é preciso repetir GET em polling contínuo.
- Ao ligar, identificar o agente por claims autenticadas, adicionar ligação ao tracker e ao grupo autorizado.
- Ao desligar, remover ConnectionId; marcar Offline apenas se já não houver outras ligações válidas.

Modelo e contratos do exercício original

Entidade e contratos C#

```
public enum AgentStatus { Offline, Online, Away, Busy }

public class Agent
{
    public Guid Id { get; set; }
    public Guid TeamId { get; set; }
    public AgentStatus Status { get; set; }
}

public interface IAgentRepository
{
    Task<Agent?> GetByIdAsync(Guid id);
    Task<bool> UpdateAsync(Agent agent);
}

public interface IAgentStatusNotifier
{
    Task NotifyStatusChangedAsync(Guid teamId, Guid agentId, AgentStatus status);
}
```

Serviço de alteração de estado

Serviço: validação, persistência e notificação

```
public async Task<bool> UpdateAgentStatusAsync(Guid id, AgentStatus status)
{
    var agent = await _repository.GetByIdAsync(id);
    if (agent is null || !IsValidManualTransition(agent, status))
        return false;

    agent.Status = status;
    if (!await _repository.UpdateAsync(agent)) return false;

    await _notifier.NotifyStatusChangedAsync(agent.TeamId, agent.Id, status);
    return true;
}

private static bool IsValidManualTransition(Agent agent, AgentStatus next)
    => next != AgentStatus.Offline && agent.Status != next;
```

Nota: a validação acima é simplificada. Se Offline→Busy/Away for proibido, acrescentar regras explícitas; se Offline→Online for causado pela ligação, separar o fluxo automático. Não confiar no id do pedido para autorizar alterações: usar a identidade autenticada. Uma falha de SignalR após SaveChanges pode exigir retries/outbox.

SignalR Hub e ConnectionTracker

Hub: ligações e grupos

```
[Authorize]
public class AgentHub : Hub
{
    private readonly ConnectionTracker _tracker;
    public AgentHub(ConnectionTracker tracker) => _tracker = tracker;

    public override async Task OnConnectedAsync()
    {
        if (!Guid.TryParse(Context.UserIdentifier, out var agentId) ||
            !Guid.TryParse(Context.User?.FindFirst("teamId")?.Value, out var teamId))
        {
            Context.Abort();
            return;
        }
        _tracker.AddConnection(agentId, Context.ConnectionId);
        await Groups.AddToGroupAsync(Context.ConnectionId, $"team-{teamId}");
        await base.OnConnectedAsync();
    }
}
```

Context.ConnectionId identifica uma ligação; Context.UserIdentifier identifica o utilizador, se o mapeamento de claims estiver configurado. Um agente pode ter vários ConnectionIds. Groups controla destinatários de mensagens; ConnectionTracker conta ligações. São mecanismos complementares.

Notificador e DI

Notificador e registos DI

```
public class SignalRAgentStatusNotifier : IAgentStatusNotifier
{
    private readonly IHubContext<AgentHub> _hub;
    public SignalRAgentStatusNotifier(IHubContext<AgentHub> hub) => _hub = hub;
    public Task NotifyStatusChangedAsync(Guid teamId, Guid agentId, AgentStatus status)
        => _hub.Clients.Group($"team-{teamId}")
            .SendAsync("AgentStatusChanged", agentId, status.ToString());
}

// Program.cs
builder.Services.AddControllers();
builder.Services.AddSignalR();
builder.Services.AddScoped<IAgentRepository, AgentRepository>();
builder.Services.AddScoped<IAgentService, AgentService>();
builder.Services.AddScoped<IAgentStatusNotifier, SignalRAgentStatusNotifier>();
builder.Services.AddSingleton<ConnectionTracker>();
// AddDbContext, AddAuthentication/JWT and AddAuthorization configured separately
app.MapControllers();
app.MapHub<AgentHub>("/agentHub");
```

Casos de falha, escala e segurança

Pontos avançados para discutir

- Concorrência: remover a última ligação e marcar Offline deve ser coordenado com reconexões simultâneas; ConcurrentDictionary isolado não torna toda a operação atómica.
- Múltiplas instâncias Kubernetes: um singleton local não vê ligações de outros pods. Usar store partilhada (ex.: Redis) e mecanismo de distribuição de eventos SignalR.
- Heartbeats/TTL: desconexões abruptas demoram a ser detetadas; definir política de expiração e múltiplas sessões.
- Autorização: teamId deve ser derivado de claims validadas ou de lookup seguro; nunca confiar cegamente num teamId enviado pelo cliente.
- Persistência e notificação: DB é fonte de verdade do estado persistido; para garantias de entrega, considerar outbox/retries.

Divisão de tarefas por equipa

- Backend A — Entidade, enum, regras de transição, serviço e testes unitários.
- Backend B — Controllers, DTOs, autenticação/autorização e contratos HTTP.
- Backend C — Repository EF Core, migrations, consultas por equipa, testes de integração.
- Real-time — Hub, grupos SignalR, notificador, presença e desconexões.
- Frontend/QA — Ligação SignalR, interface de estados, testes de reconexão, carga e permissões.

Pontos de atenção durante o live coding

- O agente pode ter várias tabs/dispositivos: Offline apenas após a última ligação válida.
- O sistema deve impedir que um agente altere o estado de outro sem permissão.
- A pertença ao grupo SignalR é determinada com base em informação validada pelo servidor.
- O ConnectionTracker em memória não se estende a vários pods Kubernetes; não resolve, por si só, corridas entre disconnect e reconnect.
- A leitura inicial vem de um GET/snapshot; SignalR publica as alterações posteriores.

Perguntas de entrevista — Português e English

PT Porque usar SignalR em vez de polling?

Resposta: Envia atualizações imediatamente a clientes ligados e evita pedidos HTTP repetidos.

EN Why use SignalR instead of polling?

Answer: It pushes changes to connected clients and avoids repeated polling requests.

PT Porque precisamos de ConnectionTracker e Groups?

Resposta: O tracker conta ligações por agente; Groups define quem recebe notificações.

EN Why do we need both ConnectionTracker and Groups?

Answer: The tracker records each agent's connections; Groups determine notification recipients.

PT O que acontece quando se fecha apenas uma de duas tabs?

Resposta: O agente continua ligado; não deve passar a Offline.

EN What happens when one of two tabs closes?

Answer: The agent stays connected; do not mark them Offline.

PT Como escalarias para vários pods?

Resposta: Presença partilhada (por exemplo Redis), gestão de concorrência e distribuição SignalR entre instâncias.

EN How would you scale to multiple pods?

Answer: Use a shared presence store, handle concurrency, and scale SignalR messaging across instances.

PT Que fazer se a notificação falhar após gravar na BD?

Resposta: Manter a BD como fonte de verdade; considerar retries, reconciliação e outbox.

EN What if a notification fails after the database update?

Answer: Keep the database as the source of truth; consider retries, reconciliation, and an outbox.

Docker, containers e Kubernetes

Conceitos operacionais que surgem em entrevistas backend.

Docker — image, container e Dockerfile

Docker — revisão rápida

Image = template com aplicação e dependências; container = instância executável. Dockerfile descreve build; docker build cria imagem; docker run cria/inicia container. COPY copia ficheiros; RUN executa instruções de build; EXPOSE documenta porta, não a publica. Docker Compose junta serviços. Volumes persistem dados. Em Compose, usar hostname do serviço (ex.: db), não localhost para outro container. Multi-stage build reduz tamanho final. Kubernetes orquestra containers; não requer Docker Engine como runtime.

```
docker build -t agent-api .
docker run -d -p 8080:8080 agent-api
docker ps
docker logs <container>
```

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY . .
RUN dotnet publish -c Release -o /out

FROM mcr.microsoft.com/dotnet/aspnet:8.0
WORKDIR /app
COPY --from=build /out .
EXPOSE 8080
ENTRYPOINT ["dotnet", "AgentApi.dll"]
```

Multi-stage builds separam SDK de compilação e runtime. EXPOSE documenta uma porta; docker run -p publica a porta. Em Compose, os serviços encontram-se por nomes DNS na rede da stack, não por localhost.

Volumes, rede e orquestração

- Volume: mantém dados para além do ciclo de vida de um container, importante para bases de dados.
- Docker Compose: descreve serviços, redes e volumes num ficheiro YAML para desenvolvimento e testes locais.
- Kubernetes: gere deployments, scaling, health checks e substituição de containers; não substitui a necessidade de gestão de estados partilhados.

- Logs: docker logs e observabilidade estruturada; cuidar de secrets, execução como utilizador não-root e imagens atualizadas.

Perguntas de entrevista — Português e English

PT Qual é a diferença entre image e container?

Resposta: Image é um pacote/template; container é uma instância dessa imagem.

EN What is the difference between an image and a container?

Answer: An image is a packaged template; a container is an instance of it.

PT O que fazem docker build e docker run?

Resposta: Build cria a imagem; run cria e inicia o container.

EN What do docker build and docker run do?

Answer: Build creates an image; run creates and starts a container.

PT Porque usar multi-stage builds?

Resposta: Para não incluir o SDK completo na imagem final de runtime.

EN Why use multi-stage builds?

Answer: To keep build tools out of the final runtime image.

PT Porque localhost falha entre containers Compose?

Resposta: Cada container tem o seu próprio localhost; usa-se o nome do serviço na rede.

EN Why does localhost fail between Compose containers?

Answer: Each container has its own localhost; use the service name on the shared network.

Live Coding — exercícios resolvidos

Soluções extraídas do material de preparação e pontos a explicar ao entrevistador.

FizzBuzz

```
for (int i = 1; i <= 100; i++)
{
    if (i % 3 == 0 && i % 5 == 0) Console.WriteLine("FizzBuzz");
    else if (i % 3 == 0) Console.WriteLine("Fizz");
    else if (i % 5 == 0) Console.WriteLine("Buzz");
    else Console.WriteLine(i);
}
```

Reverse string

```
static string Reverse(string input)
{
    var chars = input.ToCharArray();
    Array.Reverse(chars);
    return new string(chars);
}
```

Palindrome

```
static bool IsPalindrome(string text)
{
    int left = 0, right = text.Length - 1;
    while (left < right)
        if (text[left++] != text[right--]) return false;
    return true;
}
```

Find duplicates

```
static List<int> Duplicates(IEnumerable<int> numbers)
{
    var seen = new HashSet<int>();
    var duplicates = new HashSet<int>();
    foreach (var n in numbers)
        if (!seen.Add(n)) duplicates.Add(n);
    return duplicates.ToList();
```

```
}
```

Second-largest distinct value

```
static int SecondLargest(IEnumerable<int> values)
{
    var top = values.Distinct().OrderByDescending(x => x).Take(2).ToArray();
    if (top.Length < 2) throw new ArgumentException("Need two distinct values");
    return top[1];
}
```

Character frequency

```
static Dictionary<char, int> CountCharacters(string input)
{
    var counts = new Dictionary<char, int>();
    foreach (char c in input)
    {
        if (counts.ContainsKey(c)) counts[c]++;
        else counts[c] = 1; // equivalent to Add(c, 1) for a new key
    }
    return counts;
}
```

Two Sum — return indices

```
static int[] TwoSum(int[] values, int target)
{
    var seen = new Dictionary<int, int>();
    for (int i = 0; i < values.Length; i++)
    {
        int missing = target - values[i];
        if (seen.TryGetValue(missing, out int j)) return new[] { j, i };
        seen[values[i]] = i;
    }
    return Array.Empty<int>();
}
```

LINQ — filtering and grouping

```
var expensive = products.Where(p => p.Price > 100)
                        .OrderBy(p => p.Price).ToList();

var totals = orders.GroupBy(o => o.CustomerId)
    .Select(g => new { CustomerId = g.Key, Total = g.Sum(o => o.Amount) });
```

REST CRUD — exemplo de endpoints

```
app.MapGet("/products", (IProductService service) => service.GetAllAsync());
// Em POST: validar DTO, chamar service, devolver 201 Created com Location.
// Em PATCH: validar identidade, regras e concorrência, devolver 200/204 ou 404/409.
```

Melhorias e casos limite a explicar

- FizzBuzz: testar múltiplos de 3 e 5 antes das condições individuais.
- Reverse: strings .NET têm UTF-16; inverter chars pode não preservar todos os grafemas Unicode. Para entrevistas básicas, explicitar o pressuposto ASCII.
- Palindrome: clarificar se maiúsculas, espaços e pontuação contam.
- Duplicates: HashSet melhora a procura média; testar entradas vazias.
- SecondLargest: definir comportamento com menos de dois valores distintos.
- CharacterFrequency: Dictionary<char,int> não trata automaticamente diferenças de case.
- TwoSum: distinguir valores de índices e usar Dictionary para O(n) médio.
- CRUD: validar DTO, permissões, HTTP status e cancellation tokens.

Perguntas de entrevista — Português e English

PT Qual a complexidade média de Two Sum com Dictionary?

Resposta: O(n) no tempo e O(n) na memória.

EN What is the average complexity of Two Sum using a Dictionary?

Answer: O(n) time and O(n) extra space.

PT Porque é importante clarificar os requisitos antes do código?

Resposta: Para definir entrada, saída, erros, casos limite e critérios de aceitação.

EN Why clarify requirements before coding?

Answer: To establish inputs, outputs, errors, edge cases, and acceptance criteria.

PT Porque usar TryParse para entrada do utilizador?

Resposta: Porque evita exceção normal de parsing quando o input é inválido.

EN Why use TryParse for user input?

Answer: It handles invalid input without throwing a parsing exception.

Estratégia de entrevista e revisão final

Como apresentar decisões, explicar compromissos e distribuir tarefas.

- Clarificar requisitos antes de desenhar: número de utilizadores, latência, tempo real, permissões, disponibilidade e persistência.
- Começar pela solução mais simples que satisfaz o problema; justificar quando Redis, filas ou microserviços são necessários.
- Descrever primeiro o fluxo de dados e os contratos, depois as classes e os detalhes técnicos.
- Durante live coding, confirmar entrada/saída, casos limite, erros, concorrência e complexidade.
- Explicar trade-offs: simplicidade vs escalabilidade, consistência vs disponibilidade, leitura vs custo de escrita.

Perguntas iniciais em inglês

“Do we need real-time updates or is polling acceptable?” | “Can agents see everyone or only their teammates?” | “How many concurrent connections do we expect?” | “Who can change an agent’s status?”

Roteiro de resposta para System Design

1. 1. Perguntar sobre requisitos funcionais e não funcionais (tempo real, escala, equipas, autorização).
2. 2. Definir os recursos e contratos HTTP.
3. 3. Desenhar camadas e direção das dependências.
4. 4. Explicar GET inicial, PATCH e notificações SignalR.
5. 5. Tratar reconexões, concorrência, falhas e testes.
6. 6. Dividir tarefas e justificar a sequência de implementação.

Checklist de revisão

7. Checklist final de preparação

- Conseguir desenhar Controller → Service → Repository → DB e a via de notificação SignalR.
- Explicar porque o Hub fica na API e a interface do notifier na Application.
- Escrever enum AgentStatus, Agent, IAgentRepository e UpdateAgentStatusAsync sem consultar apontamentos.
- Saber distinguir UserIdentifier, ConnectionId e teamId.
- Explicar múltiplas tabs, timeout, reconexão e limitação do tracker Singleton.
- Resolver FizzBuzz, Two Sum, contagem de caracteres e LINQ básico.
- Explicar DI lifetimes, HTTP codes, async/await, JWT e índices.
- Descrever Docker image vs container, Dockerfile, Compose e Kubernetes.
- Ter preparada uma resposta em inglês de 90 segundos para o cenário de agentes.

8. Checklist de entrevista

- Saber identificar métodos seguros e idempotentes.
- Explicar PUT vs PATCH sem afirmar que PATCH é sempre idempotente.
- Distinguir efeito no servidor de código de resposta.
- Saber escolher 200, 201 e 204.
- Distinguir 400, 401, 403, 404, 409 e 500.
- Saber explicar stateless, URIs de recursos, autenticação e autorização.
- Aplicar os conceitos à Agent API e justificar cada endpoint.

Perguntas de entrevista — Português e English

PT Porque escolher inicialmente um monólito modular?

Resposta: Menor complexidade operacional, deploy simples e fronteiras claras para evoluir.

EN Why start with a modular monolith?

Answer: It offers simpler deployment and operations while keeping clear module boundaries.

PT Como distribuirias o trabalho?

Resposta: Contratos e regras em conjunto; tarefas separadas para domínio/serviços, API/segurança, persistência, SignalR e testes.

EN How would you divide the work?

Answer: Agree on contracts first, then split domain/services, API/security, persistence, SignalR, and testing.

PT Como justificas uma decisão técnica?

Resposta: Explico requisitos, alternativa, benefício, custo e condições para rever a escolha.

EN How do you justify a technical decision?

Answer: Explain requirements, alternatives, benefits, trade-offs, and when to revisit the choice.

Nota final sobre a origem do material

Este documento consolida os conteúdos e exemplos do guia de preparação anterior e acrescenta esclarecimentos técnicos e traduções para fins didáticos. O cenário Agent Presence System foi trabalhado como simulação; não é possível atribuir todas as questões aqui incluídas a uma entrevista real específica.